MURU-BENCH: A Benchmark for Mathematical Reasoning Under Uncertainty

Swetank Kumar
Department of Computer Science
SRM Institute of Science and Technology (SRMIST)
Chennai, Tamil Nadu, India
swetankkumar391@gmail.com
https://github.com/swetank18

Abstract

Mathematical-reasoning benchmarks score correctness as a single number, but the mathematics that matters in practice—epidemiology, reliability engineering, decision support—is uncertain by construction. A benchmark that ignores stated confidence cannot distinguish a calibrated reasoner from a confident guesser. We introduce MURU-BENCH (**M**athematical **R**easoning **U**nder **U**ncertainty **Bench**mark), a 3,000-problem benchmark where every prediction is scored on four orthogonal dimensions: point estimate, confidence-interval coverage, stated-confidence calibration, and reasoning-framework identification. Problems span five categories—Bayesian Updating, Conditional Probability Chains, Distribution Estimation, Decision Under Uncertainty, and Adversarial Ambiguity—and five difficulty levels, generated by 21 parametric templates with closed-form solutions, so correctness is guaranteed by construction and the dataset is reseedable to arbitrary scale. The harness reports six metrics including Expected Calibration Error and a safety-relevant overconfidence rate, and ships a unified API client for six hosting providers.

We first validate the harness with five simulated capability tiers: paired McNemar tests detect every adjacent-tier gap at $p < 10^{-3}$, including the smallest configured gap of 11.6 percentage points, establishing that the 301-problem test split has statistical power to discriminate tier-scale differences. We then report the first language-model leaderboard on MURU-BENCH, evaluating five hosted open-weights models. Headline Accuracy@CI spans 51 percentage points (Llama-3.1-8B: 43.1%; GPT-OSS-120B: 94.5%, partial coverage). Across the panel, accuracy and calibration are tightly coupled (Spearman $\rho = -0.90$ between Accuracy@CI and ECE; Pearson $r = -0.99$; $n = 5$ models, to be expanded in follow-on work): the safety-relevant overconfidence rate falls by an order of magnitude—from 51.4% to 5.5%—across the same accuracy range. Two otherwise-strong models drop to $\leq 30\%$ on Decision-Under-Uncertainty problems while scoring 90–100% elsewhere, exposing a category-shaped capability hole that single-number leaderboards cannot surface. Benchmark, harness, raw responses, statistical scripts, and CI are released at https://github.com/swetank18/MURU_proj under CC-BY-4.0 (data) and MIT (code).

1 Introduction

Recent progress on mathematical benchmarks has been startling. Frontier language models match strong human performance on grade-school word problems [1], hold their own against competition-level mathematics [2], and span a wide range of subject areas at adult-expert level [3]. These benchmarks share a structural feature: each problem has a single deterministic answer. There is no ambiguity in the ground truth, and a response is either right or wrong as a string of characters.

The mathematical reasoning encountered in practice almost never has this shape. A clinician interpreting a diagnostic test combines an uncertain prevalence with imperfect test characteristics. A reliability engineer chains together conditional probabilities estimated from limited data. A policy analyst weighs courses of action against utilities that

depend on states whose probabilities are themselves estimated. In each case, the answer is not a number but a distribution or an interval, and the quality of the reasoning depends as much on the stated confidence as on the central estimate.

The calibrated uncertainty gap. We argue that the standard evaluation paradigm for language-model mathematics has a load-bearing blind spot: it provides no signal about whether a model knows when it is uncertain. We call this the *calibrated uncertainty gap*. The gap matters operationally, not just academically. A medical-reasoning assistant that returns a confidently incorrect posterior probability is more dangerous than one that flags its own uncertainty. A financial-risk model that reports a point estimate understates tail risk by construction. A policy model that anchors on the most salient figure in a problem stem reproduces the base-rate fallacy at scale. A benchmark that only rewards point accuracy on deterministic problems supplies no metric that can be optimised against these failure modes, because the failure modes are invisible to it.

Contributions. This paper makes four contributions.

1. A 3,000-problem benchmark across five categories of mathematical uncertainty. Each problem ships with a ground-truth point estimate, a confidence interval at a stated level, an ordered list of solution steps, the required reasoning framework, and an enumeration of common failure modes (Section 3). All 3,000 problems are generated by 21 parametric templates with closed-form solutions, so correctness is guaranteed by construction and the benchmark can be expanded by changing the seed.
2. A four-dimensional evaluation protocol (Section 3.2) that scores point accuracy, interval coverage, metacognitive calibration, and framework correctness as orthogonal capabilities, instantiated as six metrics that include Expected Calibration Error and an overconfidence rate (Section 4).
3. A harness-validation study (Section 5) using five simulated capability tiers spanning a 70-percentage-point accuracy range. Percentile bootstrap 95% intervals on every metric ($B = 10{,}000$) and paired McNemar tests on every adjacent-tier comparison establish that the metric implementations recover the intended ordering and that the held-out test split ($n = 301$) has the statistical power to discriminate tier-scale capability differences as small as 11.6 percentage points ($p < 10^{-3}$ throughout).
4. The first language-model leaderboard on MURU-BENCH (Section 6). We report test-split measurements for five hosted open-weights models—Llama-3.1-8B, Llama-3.3-70B, Llama-4-Scout-17B, GPT-OSS-120B, and Qwen3-32B—with paired-bootstrap 95% intervals on every metric. The leaderboard shows a 51-percentage-point spread in Accuracy@CI on the same problem set, recovers the predicted accuracy/calibration coupling at $\rho = -0.90$, and surfaces a category-shaped capability hole on Decision-Under-Uncertainty problems that single-number leaderboards on existing benchmarks would not detect.

Reproducibility and extensibility. The harness includes a unified API client for six hosting providers; deterministic generation under a fixed seed; stratified train, validation, and test splits; a 26-test pytest suite that covers metrics, response parsing, and dataset invariants; a continuous-integration configuration that runs the suite on Python 3.10, 3.11, and 3.12 on every push; a Datasheet for Datasets [15]; and a Croissant [16] metadata file (Appendices G and I). Each row of the leaderboard is reconstructible bit-for-bit from a single per-model JSON archive in `evaluation/baselines/`, which holds the raw model responses, the parsed predictions, and the computed metrics for every problem in the run.

2 Related Work

Mathematical reasoning benchmarks. GSM8K [1] tests grade-school arithmetic, has a single deterministic answer per problem, and has been largely saturated by frontier models. MATH [2] extends to competition-level problems across algebra, geometry, and number theory, but each problem still admits a unique numerical or symbolic answer. BIG-Bench Hard [4] broadens the task distribution but contains no formal probability problems. MathBench [12] is hierarchical across education levels but does not require uncertainty quantification. TheoremQA [13] probes theorem application against deterministic targets. None of these benchmarks ask a model to produce a calibrated confidence interval as part of the answer; the ground-truth label is always a single object.

Table 1: Comparison of MURU-BENCH with existing mathematical-reasoning benchmarks. MURU-BENCH is the only benchmark in this list that requires calibrated confidence intervals as part of the answer, that contains adversarial ambiguity by design, and that scores calibration directly. ✓ = yes, ✗ = no, ∼ = partial.

Benchmark	Size	Math Reasoning	Uncertainty Reasoning	CI Required in Answer	Calibration Measured	Adversarial Ambiguity	Difficulty Levels
GSM8K	8.5K	✓	✗	✗	✗	✗	1
MATH	12.5K	✓	✗	✗	✗	✗	5
MMLU	15.9K	∼	✗	✗	✗	✗	1
BIG-Bench Hard	6.5K	∼	✗	✗	✗	✗	1
TheoremQA	800	✓	✗	✗	✗	✗	1
TruthfulQA	817	✗	✗	✗	✗	✓	1
MathBench	3.7K	✓	✗	✗	✗	✗	3
MURU-BENCH	**3,000**	✓	✓	✓	✓	✓	**5**

Calibration and confidence in language models. Kadavath et al. [5] showed that language models possess a partial, noisy signal of their own correctness through token probabilities. Guo et al. [6] introduced the binned Expected Calibration Error metric we adopt. Prompting strategies for verbalised confidence [7] elicit confidence values that correlate with accuracy but exhibit systematic biases, particularly overconfidence in long-tail domains. Xiong et al. [14] surveys this literature and finds that scale improves calibration on average without eliminating its failure modes. The connecting thread is that calibration is studied as a property of the model's metacognition over an existing benchmark, rather than as a target the benchmark itself measures. MURU-BENCH differs by promoting calibration to a first-class evaluation target, with ground-truth confidence intervals against which the model's stated confidence can be scored.

Uncertainty quantification. The machine-learning literature on predictive uncertainty—Bayesian neural networks [8], deep ensembles [9], conformal prediction [10]—studies how to make a learned model report uncertainty about its own predictions. MURU-BENCH tests something different and prior to that machinery: whether a model can reason about uncertainty as a mathematical concept in the first place. The skill we measure is computing posteriors, propagating parameter uncertainty through compositions, and recognising when a problem admits multiple defensible formalisations.

Adversarial and ambiguous evaluation. TruthfulQA [11] probes whether models repeat popular misconceptions when asked. The Adversarial Ambiguity category in MURU-BENCH is structurally analogous in that it deliberately stresses confident but wrong reasoning. The mechanism is different: TruthfulQA exploits world-knowledge cues, while our adversarial problems are mathematically precise and exploit ambiguity in the formalisation rather than in the facts.

Positioning. Table 1 places MURU-BENCH alongside the closest reference benchmarks. To our knowledge, it is the first benchmark that combines formal mathematical reasoning with calibrated uncertainty quantification, requires both correct computation and calibrated confidence, and arranges problems on a structured difficulty hierarchy.

3 Dataset Construction

3.1 Design Principles

Five principles shaped what counts as a MURU-BENCH problem.

1. **Unambiguous correctness.** A trained mathematician, given the stated assumptions, would agree that the ground-truth point estimate and confidence interval are correct.

Table 2: The five MURU-BENCH categories. Each is built around a specific kind of mathematical uncertainty, from textbook Bayesian inference at one end to deliberately adversarial constructions at the other.

Category	Count	%	Core skill tested
Bayesian Updating	910	30.3	Sequential prior-to-posterior inference with uncertain likelihoods
Distribution Estimation	660	22.0	Estimating population parameters from finite samples with appropriate CIs
Decision Under Uncertainty	525	17.5	Expected utility computation with uncertain state probabilities
Adversarial Ambiguity	474	15.8	Recognising when a problem admits multiple valid formalisations
Cond. Probability Chains	431	14.4	Multi-step conditional probability with uncertain intermediates
Total	**3,000**	**100**	

2. **Measurable failure.** An overconfident response fails in a quantifiable way: either its point estimate falls outside the ground-truth interval, or its stated confidence is miscalibrated against the bin-wise accuracy of the population of comparable predictions.
3. **Mathematical uncertainty.** The uncertainty in the problem is genuinely mathematical (unknown parameters, ambiguous formalisation), not linguistic vagueness or world-knowledge uncertainty.
4. **Active reasoning.** The problem cannot be solved by retrieval or pattern matching alone; it requires applying a probabilistic framework to specific numerical inputs.
5. **Non-trivial intervals.** The ground-truth confidence interval is always non-degenerate (positive width), so the problem genuinely involves uncertainty rather than smuggling a deterministic answer in interval clothing.

3.2 Formal Problem Definition

A MURU-BENCH problem p consists of a natural-language stem s_p and a ground-truth tuple $(y_p^*, [\ell_p, u_p], \alpha_p, f_p)$. Here y_p^* is the point estimate, $[\ell_p, u_p]$ the confidence interval at level α_p, and $f_p \in \mathcal{F}$ the correct reasoning framework drawn from

$$\mathcal{F} = \{\texttt{bayesian}, \texttt{frequentist}, \texttt{decision\_theory}, \texttt{information\_theory}, \texttt{monte\_carlo}\}.$$

A model $\mathcal{M}$ receives s_p and produces a response tuple $(\hat{y}_p, [\hat{\ell}_p, \hat{u}_p], c_p, \hat{f}_p)$, where $c_p \in [0, 1]$ is the stated confidence. The metrics in Section 4 are functions of the ground-truth and response tuples across the test set $\mathcal{T} = \{p_1, \ldots, p_N\}$. The formalisation is what makes MURU-BENCH a four-output evaluation rather than a single-number one: every prediction is scored on point accuracy, interval coverage, metacognitive calibration, and process correctness.

3.3 Problem Categories

We organise problems into five categories, each isolating a distinct kind of mathematical uncertainty. Table 2 lists the counts.

Bayesian Updating (910 problems). At lower difficulty, problems require a single application of Bayes' theorem (medical diagnostic tests, quality-control screening). At higher difficulty, the prior, the likelihood ratio, or both become uncertain, and the problem demands sequential updating or hierarchical shrinkage with non-trivial intermediate posteriors.

Conditional Probability Chains (431 problems). Multi-step chains in which one or more transition probabilities are uncertain. Templates include sequential pipelines (multi-stage manufacturing), parallel redundancy under common-cause failure, and branching cascades with dependent failure modes. The hard part is propagating uncertainty across multiple conditioning steps without collapsing the joint to a point.

Distribution Estimation (660 problems). Inference about population parameters from finite samples. Templates cover sample-mean estimation with t-distribution intervals, Wilson intervals for proportions, χ^2 intervals for variances, and two-component mixture decomposition. Higher-difficulty variants add stratification uncertainty and multimodality.

Decision Under Uncertainty (525 problems). Expected-value and expected-utility computation under uncertain state probabilities, uncertain outcome values, or both. Templates include multi-option decisions with sensitivity analysis, sequential decisions with value-of-information, and insurance pricing in which Poisson frequency multiplies an uncertain severity distribution. These problems test whether models compose two independent uncertainties correctly rather than handling one and dropping the other.

Adversarial Ambiguity (474 problems). Problems built to provoke overconfident reasoning. Templates include Simpson's-Paradox-style aggregate-versus-subgroup conflicts, reference-class problems in which the answer depends on which reference population is used, and anchoring constructions that present a misleading prior alongside data. These problems do not have a single defensible point estimate; the ground truth is an interval over the defensible formalisations.

3.4 Parametric Generation

Hand-crafting 3,000 problems would limit scale, induce stylistic bias, and tie correctness to whoever wrote the problem. We instead implement 21 *parametric generation templates*. Each template specifies:

- a mathematical structure (e.g., "sequential Bayesian updating with k evidence items and uncertain likelihood ratios");
- a parameter space (e.g., prior $\in [0.01, 0.5]$, likelihood ratio $\in [2, 50]$);
- a set of domain contexts (medical testing, quality control, forensic evidence) sampled uniformly to diversify the surface form;
- a closed-form solution function that takes sampled parameters to a ground-truth point estimate, confidence interval, and ordered list of solution steps;
- difficulty modifiers (extra uncertain parameters, deeper inference chains) that lift a template into higher difficulty bands.

Table 14 in Appendix B lists all 21 templates with their categories and difficulty ranges. Three properties follow from the parametric construction: numerical diversity within a template (no two instantiations share parameters), correctness by construction (the solution is computed, not estimated), and unbounded scalability (additional problems can be sampled by varying the seed).

3.5 Difficulty Calibration

Each problem is assigned a difficulty level from 1 to 5. The level is determined by the number of uncertain parameters, the depth of the inference chain, and whether adversarial structure is present (Table 3).

The difficulty distribution (Figure 2 in Appendix A) is roughly bell-shaped around D3, with D5 problems deliberately the smallest band (271, 9.0%) because they are the most expensive to validate and the most variable in the size of their defensible-answer interval.

Table 3: Difficulty-level specification. Each level is defined by the count of uncertain parameters, the depth of the inference chain, and whether adversarial structure is present.

Level	Description	Uncertain Params	Chain Depth	Adversarial?
D1	Single-formula application	0–1	1	No
D2	Two-step reasoning	1	1–2	No
D3	Multi-step with one uncertain param	1–2	2–3	Sometimes
D4	Compound uncertainty propagation	2–3	3+	Sometimes
D5	Multi-step + adversarial + compound	3+	3+	Yes

3.6 Problem Format and Schema

Each problem is stored as a JSON document with the following fields:

- `id`: unique identifier (e.g., `MURU-0001`);
- `stem`: natural-language statement with embedded quantitative data;
- `category`: one of the five categories;
- `difficulty`: integer 1–5;
- `uncertainty_type`: `epistemic_prior`, `parameter_estimation`, etc.;
- `required_framework`: the mathematically appropriate framework, drawn from $\mathcal{F}$;
- `ground_truth`: object with `answer`, `point_estimate`, `confidence_interval`, and `ci_level` (typically 0.95);
- `solution_steps`: ordered list of reasoning steps;
- `common_failure_modes`: typical errors a model might make on this problem;
- `metadata`: author, review status, source template, and creation timestamp.

3.7 Quality Assurance Pipeline

Every problem passes through a three-stage validation pipeline.

Schema validation. An automated check confirms that all required fields are present, types are correct, the difficulty is in $\{1, \ldots, 5\}$, the category is from the allowed set, and the confidence interval is well-formed ($\text{CI}_{\text{lower}} < \text{CI}_{\text{upper}}$).

Semantic consistency. A second automated pass confirms mathematical consistency: the point estimate lies within the confidence interval; the interval is non-degenerate and not pathologically wide; the stated CI level matches the level used in the solution; the solution steps reference the correct mathematical operations.

Spot-check review. Random samples drawn from each (category, difficulty) cell are reviewed manually. A problem fails the review if the stem is unclear, the solution is wrong, or the listed failure modes are unrealistic. Failures are regenerated rather than patched.

After validation, all 3,000 problems pass the pipeline (a 100% pass rate after a single round of bug fixes in the value-of-information template, documented in Section D).

3.8 Data Splits

We split the dataset into train, validation, and test sets by stratified sampling on (category, difficulty), ensuring balanced representation across all 25 cells of the cross-product:

Table 4: Data-split sizes with stratification by category and difficulty.

Split	Count	Purpose
Train	2,398	Model fine-tuning (future work)
Validation	301	Hyperparameter selection
Test	301	Baseline evaluation (reported results)

4 Evaluation Methodology

4.1 Metrics

Six metrics, listed in Table 5, score four orthogonal dimensions of uncertainty reasoning.

Table 5: The MURU-BENCH evaluation metrics. The four dimensions are correctness (was the answer right?), calibration (does stated confidence match accuracy?), process quality (did the model identify the right framework?), and safety (does the model recognise what it does not know?).

Metric	Dimension	Range	Description
Accuracy@CI	Correctness	$[0, 1] \uparrow$	Answer falls within ground-truth CI
ECE	Calibration	$[0, 1] \downarrow$	Expected Calibration Error
Overconfidence	Safety	$[0, 1] \downarrow$	Fraction of high-confidence wrong answers
Framework Match	Process	$[0, 1] \uparrow$	Correct reasoning framework identified
Cat. breakdown	Diagnostic	—	Per-category accuracy decomposition
Diff. scaling	Diagnostic	—	Per-difficulty accuracy decomposition

Accuracy@CI. A prediction is correct iff the model's point estimate $\hat{y}$ satisfies $\mathrm{CI}_{\text{lower}} \leq \hat{y} \leq \mathrm{CI}_{\text{upper}}$, where $[\mathrm{CI}_{\text{lower}}, \mathrm{CI}_{\text{upper}}]$ is the ground-truth interval. This is strictly stricter than exact-match accuracy: the model has to produce a numerically reasonable estimate, not just write down the right formula.

Expected Calibration Error. We use the standard binned ECE of Guo et al. [6],

$$\mathrm{ECE} = \sum_{m=1}^{M} \frac{|B_m|}{N} \left|\mathrm{acc}(B_m) - \mathrm{conf}(B_m)\right|, \tag{1}$$

where B_m is the set of predictions whose stated confidence falls in the m-th bin, $\mathrm{acc}(B_m)$ is the accuracy in that bin, and $\mathrm{conf}(B_m)$ is the mean stated confidence. We use $M = 10$ equal-width bins. A perfectly calibrated model has $\mathrm{ECE} = 0$.

Overconfidence rate. The fraction of predictions on which the model states confidence above 0.7 but the answer is wrong: $\mathrm{OvConf} = |\{i : c_i > 0.7 \wedge \hat{y}_i \notin \mathrm{CI}_i\}|/N$. This is the most safety-relevant metric in the harness, because every overconfident wrong answer is precisely the kind of output a downstream decision-support system should not propagate.

Framework match. The fraction of predictions on which the model identifies the correct framework. This separates "the model can compute the answer" from "the model knows why this is the right computation."

4.2 Prompting Protocol

Every model receives a structured system prompt instructing it to (i) identify the correct framework from five options, (ii) show step-by-step reasoning, (iii) report a point estimate as a single number, (iv) report a confidence interval, and

Table 6: Harness output across five *simulated* responder tiers on the test set ($n = 301$), with percentile bootstrap 95% intervals (10,000 resamples). Tier behaviour is specified by the simulator; the table verifies that all four headline metrics recover the intended ordering with adjacent-tier intervals that are non-overlapping or near-non-overlapping, and that the tier separation exceeds within-tier sampling noise. These numbers reflect simulator configuration, not any LLM.

Model Tier	**Acc@CI** (↑)	**ECE** (↓)	**OvConf** (↓)	**FwMatch** (↑)
Random Baseline	7.3% [4.7, 10.3]	0.515 [0.483, 0.547]	36.2% [30.9, 41.5]	33.9% [28.6, 39.2]
Heuristic Baseline	31.2% [25.9, 36.5]	0.470 [0.418, 0.522]	44.5% [38.9, 50.2]	47.2% [41.5, 52.8]
Competent Model	49.2% [43.5, 54.8]	0.239 [0.189, 0.294]	21.6% [16.9, 26.2]	67.1% [61.8, 72.4]
Strong Model	60.8% [55.1, 66.1]	0.178 [0.132, 0.231]	20.3% [15.9, 24.9]	83.7% [79.4, 87.7]
Expert Model	77.1% [72.1, 81.7]	0.183 [0.151, 0.225]	9.6% [6.6, 13.0]	89.0% [85.4, 92.4]

(v) state a confidence probability between 0 and 1. Responses are parsed by regular expressions into four structured fields (`FRAMEWORK`, `POINT_ESTIMATE`, `CONFIDENCE_INTERVAL`, `CONFIDENCE`); the temperature is set to 0 throughout. Full prompt text appears in Appendix E.

4.3 Model-Agnostic Evaluation API

The harness (`evaluation/run_eval.py`) supports six API backends—OpenAI, Anthropic, Google, Groq, OpenRouter, and Together AI—and dispatches to the right backend by inspecting the model-name prefix. Every run is persisted as a JSON archive containing the timestamp, the model identifier, the raw API response for every problem, the parsed predictions, and the computed metrics. That archive is the unit of reproducibility for every row of the leaderboard in Section 6.

5 Harness Validation via Simulated Capability Tiers

Why this section is here. Before reporting language-model measurements, we want to know that the harness behaves correctly under controlled inputs. We feed the harness five *simulated* responder profiles whose accuracy and calibration properties are specified by the simulator and ask three questions. Do the metric implementations recover the configured ordering on every metric? What is the smallest tier-scale capability gap the test split can resolve at $\alpha = 0.05$? Do the per-difficulty and per-category decompositions separate responders along the dimensions they are designed to probe? The simulator is, in effect, a unit test of the harness; the numerical values in this section are functions of the simulator profiles, not of any language model.

Simulator construction. Each tier is a `random.Random` instance parameterised by a per-difficulty success probability and a confidence-distribution shape. The five tiers (Random, Heuristic, Competent, Strong, Expert) span a 70-percentage-point accuracy range; the range is chosen to bracket plausible real-world responders rather than to estimate any specific model. Source: `evaluation/run_baselines.py`, function `MODEL_PROFILES`; bootstrap and McNemar analysis are reproduced bit-for-bit by `evaluation/bootstrap_analysis.py` with seed 42 and 10,000 resamples.

5.1 Discriminability of the Harness

Table 6 shows the harness output across all five simulated tiers, with percentile bootstrap 95% intervals on every metric ($B = 10{,}000$ resamples). The tiers were configured to be monotone in accuracy and calibration. The harness is functioning correctly iff it recovers that ordering on every metric and the bootstrap intervals separate the tiers; failure of either condition would indicate a metric-implementation bug or insufficient statistical power.

Table 7: Paired McNemar comparisons of adjacent-tier Accuracy@CI on the test set ($n = 301$, exact two-sided binomial). Every adjacent gap is significant at $p < 10^{-3}$. The smallest detectable gap is the Competent→Strong transition at $\Delta = 11.6$ percentage points; the Random→Expert end-to-end comparison gives the dynamic range of the harness.

Lower (a)	**Higher (b)**	**ΔAcc**	$n_{b\text{-only}}$	$n_{a\text{-only}}$	p-value
Random	Heuristic	+23.9 pp	84	12	1.8×10^{-14}
Heuristic	Competent	+17.9 pp	85	31	5.4×10^{-7}
Competent	Strong	+11.6 pp	77	42	1.7×10^{-3}
Strong	Expert	+16.3 pp	78	29	2.4×10^{-6}
Random	Expert	+69.8 pp	213	3	3.2×10^{-59}

Table 8: Simulator output by difficulty level. Each per-tier curve reflects the difficulty-specific success probability set in the simulator profile, not measurements of any LLM.

Model Tier	**D1**	**D2**	**D3**	**D4**	**D5**	**D1−D5 Gap**
Random Baseline	12%	7%	6%	10%	0%	12 pp
Heuristic Baseline	65%	53%	18%	9%	0%	65 pp
Competent Model	81%	72%	44%	22%	4%	77 pp
Strong Model	88%	78%	63%	33%	14%	74 pp
Expert Model	96%	91%	81%	64%	21%	75 pp

Paired discriminability via McNemar's test. Every tier sees the same 301 problems, so paired comparisons recover more power than the unpaired-CI overlap criterion. Table 7 reports McNemar's exact two-sided test for each adjacent-tier pair on Accuracy@CI: $n_{b\text{-only}}$ counts problems on which the higher-capability tier b is correct and the lower-capability tier a is wrong, and $n_{a\text{-only}}$ counts the reverse.

What the validation establishes. The bootstrap intervals and McNemar tests jointly establish four properties of the harness. **(1)** Accuracy@CI is monotone in tier capability across the simulator's full range; the smallest configured gap (Competent→Strong, $\Delta = 11.6$ pp) is detected at $p = 1.7 \times 10^{-3}$, so the test split has the statistical power to discriminate tier-scale capability differences. **(2)** ECE decreases as the simulator's confidence distribution tightens around its accuracy; the Strong and Expert ECE intervals overlap because their confidence distributions are close by construction. This is informative for power planning: an ECE comparison between two real models in this regime would need either the validation split, the train split, or paired bootstrap on the test split. **(3)** The overconfidence-rate metric is monotone in the simulator's high-confidence-error rate, with the Heuristic interval [38.9%, 50.2%] strictly above the others; this confirms that the metric distinguishes calibration shape from accuracy magnitude. **(4)** Framework-match rate is monotone in the per-tier framework-selection probability, and all adjacent-tier intervals separate cleanly.

5.2 Difficulty Stratification Recovers Configured Profiles

Table 8 and Figure 4 (Appendix A) show simulator output by difficulty level. The per-tier per-difficulty accuracy is set as a hyperparameter of the simulator, so the table primarily verifies that the difficulty stratification is being applied faithfully. What real LLMs do across the difficulty axis is an empirical question and is the subject of Section 6.

The intended interpretation of the difficulty levels is summarised in Table 3. D1 and D2 are single-formula applications; D3 introduces multi-step reasoning with at least one uncertain parameter; D4 introduces compound uncertainty propagation; D5 layers adversarial ambiguity onto the D4 conditions. Whether a real language model exhibits a sharp transition at any specific difficulty level is an empirical question the simulator cannot answer.

Table 9: Simulator output by problem category. Differences across categories reflect simulator per-category modifiers, not measured model behaviour.

Model Tier	Adv. Ambiguity	Bayesian Upd.	Cond. Chains	Decision Unc.	Distrib. Est.
Random Baseline	0%	10%	0%	4%	17%
Heuristic Baseline	4%	43%	19%	23%	48%
Competent Model	23%	61%	47%	55%	48%
Strong Model	45%	68%	47%	62%	70%
Expert Model	49%	84%	77%	74%	91%

5.3 Category Decomposition

Table 9 decomposes simulator output by category. The simulator applies a small per-category modifier (most negative on Adversarial Ambiguity), and the harness recovers the configured ordering as expected.

We make no claim that real LLMs would rank categories in this order. The simulator's per-category modifiers are configuration choices; what real models do is what Section 6 measures.

6 Language-Model Evaluation on the Test Set

Headline takeaways. Three findings frame this section. **(i)** Accuracy@CI on the open-weights panel spans 51 percentage points (43.1%–94.5%), comfortably exceeding the 11.6-pp minimum detectable effect established for the harness in Section 5; the leaderboard ordering is therefore not bootstrap noise. **(ii)** Accuracy and calibration are tightly coupled—Spearman $\rho = -0.90$ between Accuracy@CI and ECE, Pearson $r = -0.99$ on the metric values, computed across the $n = 5$ models in this panel and to be expanded as further endpoints are evaluated. The safety-relevant overconfidence rate falls by an order of magnitude (51.4% $\rightarrow$ 5.5%) over the same accuracy range. The simulator was specifically configured to permit accuracy/calibration dissociation; real models on the panel do not exhibit it. **(iii)** A category-shaped capability hole appears on Decision-Under-Uncertainty problems: two otherwise-strong models drop to $\leq 30\%$ on this category while scoring 90–100% elsewhere, a pattern invisible to single-number leaderboards.

Scope and protocol. This section reports the first language-model measurements on MURU-BENCH. Every number is produced by the harness validated in Section 5, on the same 301-problem held-out test split, with the structured prompt template documented in Appendix E at temperature zero. We restricted the panel to free-tier hosted open-weights models on Groq and OpenRouter so that any researcher with the corresponding API key can reproduce every row at no monetary cost. Five models span the relevant capability range. Llama-3.1-8B is the small-capacity reference. Qwen3-32B and Llama-3.3-70B are mid-scale dense models. Llama-4-Scout-17B is a recent mixture-of-experts release. GPT-OSS-120B is the largest open-weights frontier-class endpoint that was available on free-tier hosting at the time of evaluation.[1] Each row is produced from a single JSON archive in `evaluation/baselines/` containing the model identifier, the run timestamp, the raw API response for every test problem, the parsed predictions, and the computed metrics. Per-metric paired-bootstrap 95% intervals use $B = 10{,}000$ resamples, the same procedure as Section 5. Parsing failures and request errors are excluded from the metric computation rather than scored as wrong, so the answered-problem coverage is below 301 where indicated.

6.1 Headline Results

The leaderboard in Table 10 spans 51 percentage points in headline Accuracy@CI between the lowest entry (Llama-3.1-8B at 43.1%, 95% CI $[37.3, 48.9]$) and the highest entry that completed enough of the test set to be ranked (GPT-OSS-120B at 94.5%, $[90.4, 97.9]$ on $n = 146$). The full-coverage range alone—41 percentage points between

[1] All five models in the panel were served via the Groq OpenAI-compatible endpoint at `https://api.groq.com/openai/v1`, queried with the OpenAI Python client. The exact Groq model identifiers are: `llama-3.1-8b-instant` (Llama-3.1-8B), `llama-3.3-70b-versatile` (Llama-3.3-70B), `meta-llama/llama-4-scout-17b-16e-instruct` (Llama-4-Scout-17B), `openai/gpt-oss-120b` (GPT-OSS-120B), and `qwen/qwen3-32b` (Qwen3-32B). The mapping between paper-display names and Groq identifiers is encoded in `MODEL_MAP` in `evaluation/run_eval.py`, so a fresh run reproduces the same endpoint targeting bit-for-bit.

Table 10: Language-model leaderboard on the MURU-BENCH test set ($n = 301$), with paired-bootstrap 95% intervals from $B = 10{,}000$ resamples. The number in parentheses after each model identifier is the answered-problem coverage; runs interrupted by Groq's free-tier daily-token cap appear below the rule and are flagged as preliminary, since their answered subsets are not random samples of the test set: the evaluations that produced these rows iterated in sorted-ID order, so the completed prefix is skewed toward easy problems (the D4+D5 share is 12–22% for the partial runs versus 29% in the full test set), which inflates their Accuracy@CI by an unquantified margin. The harness has since been updated to evaluate in a seeded-shuffled order (Appendix F) so that any future quota-truncated run yields a difficulty-representative subset.

Model	**Acc@CI** (↑)	**ECE** (↓)	**OvConf** (↓)	**FwMatch** (↑)
Llama-4-Scout-17B (300/301)	84.0% [79.7, 88.0]	0.092 [0.058, 0.136]	14.0% [10.3, 18.0]	82.3% [78.0, 86.7]
Llama-3.1-8B (276/301)	43.1% [37.3, 48.9]	0.477 [0.423, 0.542]	51.4% [45.3, 57.2]	75.4% [70.3, 80.4]
GPT-OSS-120B (146/301)	94.5% [90.4, 97.9]	0.038 [0.018, 0.074]	5.5% [2.1, 9.6]	n/a
Llama-3.3-70B (76/301)	89.5% [81.6, 96.1]	0.036 [0.022, 0.114]	9.2% [3.9, 15.8]	90.8% [84.2, 97.4]
Qwen3-32B (59/301)	71.2% [59.3, 83.1]	0.217 [0.129, 0.342]	25.4% [15.3, 37.3]	n/a

Table 11: Per-difficulty Accuracy@CI on the answered subset of each model's run. Cell counts n_d vary across rows because of differential parsing-failure and quota-cap coverage; the trend within a row is therefore interpretable without re-weighting, while across-row comparisons within a difficulty cell should be read in conjunction with the coverage column of Table 10.

Model	**D1**	**D2**	**D3**	**D4**	**D5**
Llama-4-Scout-17B	92% (n=52)	93% (n=74)	80% (n=89)	81% (n=58)	63% (n=27)
Llama-3.1-8B	39% (n=46)	52% (n=67)	44% (n=82)	41% (n=54)	30% (n=27)
GPT-OSS-120B	95% (n=37)	94% (n=35)	93% (n=42)	96% (n=26)	100% (n=6)
Llama-3.3-70B	86% (n=28)	100% (n=18)	86% (n=21)	88% (n=8)	100% (n=1)
Qwen3-32B	69% (n=13)	80% (n=20)	62% (n=16)	57% (n=7)	100% (n=3)

Llama-3.1-8B and Llama-4-Scout-17B—already exceeds the discriminable resolution that the simulator validation in Section 5 established for the harness, so the leaderboard ordering is not an artifact of bootstrap noise. Two observations frame everything that follows. The smallest model in the panel sits 11.9 points above the configured Heuristic baseline (31.2%) but 34 points below the configured Expert tier (77.1%), which means the benchmark is neither saturated by Llama-3.1-8B nor pinned to its floor. The strongest full-coverage entry (Llama-4-Scout-17B at 84.0%) sits between the Strong and Expert simulator tiers, leaving real headroom for follow-on evaluations even on a recent mixture-of-experts release.

6.2 Accuracy/Calibration Coupling

The principal empirical finding of the paper is that, across the panel we measured, accuracy and calibration are tightly coupled rather than independent. Ranking the five models by Accuracy@CI and by Expected Calibration Error gives a Spearman rank correlation of $\rho = -0.90$ (Pearson $r = -0.99$ on the metric values themselves): every gain in Accuracy@CI is accompanied by a near-monotone reduction in ECE. We report the correlation as a panel-level descriptive statistic on $n = 5$ models rather than as a population-level inferential claim, and treat it as a hypothesis to be re-tested as further endpoints (closed-weights frontier models, additional open-weights releases) are added to the leaderboard in follow-on work. Two caveats bound its strength. Only two of the five rows have full test-set coverage; the three quota-truncated rows populate the high-accuracy, low-ECE corner of the diagonal, and their Accuracy@CI is upward-biased by the easy-skewed coverage documented in Table 10, so the true slope through fully-evaluated models could be shallower. The coupling should therefore be read as suggestive rather than established until the partial rows are replaced by full-coverage re-runs. The same coupling holds for the safety-relevant overconfidence rate, which falls from 51.4% on Llama-3.1-8B to 5.5% on GPT-OSS-120B—an order-of-magnitude reduction over the same accuracy range. The coupling is not theoretically guaranteed: the four metrics are mathematically independent of one another, and the simulator was specifically configured to allow accuracy and ECE to dissociate (the Strong and Expert simu-

Table 12: Per-category Accuracy@CI for the language-model panel on the test set. Daggers (†) mark partial-coverage runs (GPT-OSS-120B $n = 146$, Llama-3.3-70B $n = 76$, Qwen3-32B $n = 59$). Category cells with very small n are flagged in the body text. The Decision-Under-Uncertainty column carries the bulk of the cross-model dispersion analysed in Section 6.4.

Model	Adv. Ambig.	Bayesian Upd.	Cond. Chains	Decision Unc.	Distrib. Est.
Llama-4-Scout-17B	67.4%	90.2%	93.0%	64.2%	97.0%
Llama-3.1-8B	36.4%	23.8%	26.3%	30.0%	92.2%
GPT-OSS-120B†	92.3%	98.0%	100.0%	73.7%	100.0%
Llama-3.3-70B†	100.0%	100.0%	100.0%	30.0%	92.9%
Qwen3-32B†	66.7%	94.1%	100.0%	0.0%	100.0%

lator tiers carry overlapping ECE intervals despite a 16-point accuracy gap, see Section 5). Real models on the panel do not exhibit that dissociation. The most natural reading is that miscalibration on MURU-BENCH is dominated by capability deficits rather than by an independent metacognitive failure mode: the reasoning errors that drive accuracy down on a given problem are also the errors that the model is most confident about. Whether a fine-tuning curriculum can decouple the two, raising calibration faster than raw accuracy, is a well-posed empirical question we leave to follow-on work.

6.3 Difficulty Scaling: Where Real Models Drop

Table 11 resolves the per-difficulty profile of each model. Llama-4-Scout-17B holds 92–93% accuracy on D1 and D2, drops to 80–81% on D3 and D4, and falls a further 18 points to 63% on D5 ($n = 27$). The largest single-step drop on its row is the D4-to-D5 transition—i.e., the addition of adversarial structure on top of compound uncertainty propagation—which is exactly the design intent for D5: to isolate adversarial overconfidence from raw computation depth. Llama-3.1-8B exhibits the opposite shape: its accuracy is approximately flat across D2–D5 (range 30–52%), suggesting that the model has not internalised the difficulty hierarchy and that its responses are dominated by surface-statistic anchoring rather than by problem-specific reasoning depth. The two partial-coverage entries (Llama-3.3-70B and GPT-OSS-120B) show no D5 drop on the very small subsets they completed before the daily-token cap, but the per-cell counts (D5 $n = 1$ and $n = 6$ respectively) are too small to interpret quantitatively.

6.4 Per-Category Profile

Table 12 decomposes Accuracy@CI by category. The single most actionable cross-model finding is that the Decision-Under-Uncertainty column is by far the most internally varied. Llama-3.3-70B drops to 30% on this category despite holding 100% on Adversarial Ambiguity and Bayesian Updating. Qwen3-32B drops to 0% on the same category (across six answered Decision-Under-Uncertainty problems) despite holding 94–100% elsewhere. GPT-OSS-120B is the only mid-tier or larger model in the panel that sustains above 70% on Decision-Under-Uncertainty (73.7% on $n = 19$); Llama-4-Scout-17B sits in the middle (64.2%). The cross-model pattern is consistent with a specific failure mode: Decision-Under-Uncertainty problems require value-of-information and expected-utility computations that compose two independent uncertainties (over states and over outcomes), and several models in the panel appear to handle each uncertainty in isolation but collapse the joint to a point estimate. This is the F2 failure catalogued in Section 7.1. Distribution Estimation, by contrast, is the easiest category for every model in the panel (all $\geq 92\%$); this category exercises a single closed-form CI computation per problem and does not require composition, which is consistent with our prior expectation that single-step formula application is well within the capability floor of every model evaluated.

6.5 Anomalies in Framework-Match Reporting

Two rows in Table 10 show a 0.0% Framework-Match rate (GPT-OSS-120B and Qwen3-32B). This is an artifact of the salvage path rather than a substantive failure. Both rows were reconstructed from progress logs after the corresponding runs were terminated mid-stream by Groq's free-tier daily-token cap, and the salvage script

(evaluation/salvage_from_log.py) recovers the parsed point estimate and stated confidence from the log but does not preserve the raw response text, so the framework field cannot be re-extracted post hoc. The point-estimate-derived metrics for these rows (Accuracy@CI, ECE, overconfidence) are valid and are reported with the same bootstrap procedure, restricted to the answered subset. We flag the framework cell as "n/a" rather than dropping the row, so a downstream reader cannot mistake a salvage-path artifact for a substantive 0% framework-recognition failure. Any future run that completes within the daily-token budget will not exhibit this issue, because the raw response is preserved in the on-disk archive at write time.

6.6 Reproducibility

Every row of Table 10 can be regenerated from a single JSON archive shipped with the repository:

1. Set the appropriate API key (GROQ_API_KEY for Groq-hosted models or OPENROUTER_API_KEY for OpenRouter-hosted models).
2. Run python evaluation/run_eval.py -model <name> -save to produce a fresh JSON archive in evaluation/baselines/.
3. Run python evaluation/aggregate_real_llm.py to recompute the bootstrap intervals and refresh paper/tables/real_llm_*.tex.

The aggregator emits one LaTeX include per table, so adding a new row to the leaderboard is a one-step operation. The full set of raw responses, parsed predictions, and computed metrics for the panel reported here is included in the submission archive, so the leaderboard is replicable bit-exactly without re-running any API calls.

7 Failure-Mode Analysis

7.1 Failure-Mode Taxonomy

The benchmark is built around four failure modes drawn from the uncertainty-reasoning literature. Each problem's common_failure_modes field records which of the four (and which template-specific variants) the problem is designed to elicit. We summarise the taxonomy here together with the patterns the language-model leaderboard in Section 6 actually exhibits.

F1: Anchoring on surface statistics. The responder fixates on the most salient number in the problem stem rather than performing the appropriate probabilistic computation. In Bayesian Updating problems the canonical instance is reporting the test sensitivity (e.g. 95%) instead of the posterior probability, which is the standard base-rate-neglect fallacy. The Llama-3.1-8B row in Table 12 (Bayesian Updating accuracy of 23.8%, the lowest in the panel) is consistent with this failure mode dominating its responses on Bayesian problems, even as it scores 92.2% on Distribution Estimation problems where no base rate is involved.

F2: Ignoring compound uncertainty. D4 and D5 problems propagate uncertainty through several layers (uncertain prior, uncertain likelihood, uncertain sample size). A responder that handles each step in isolation but collapses the joint to a point estimate produces a directionally correct answer with a catastrophically narrow confidence interval; the Accuracy@CI metric flags this when the point estimate falls outside the ground-truth interval. The Decision-Under-Uncertainty column in Section 6.4 (Llama-3.3-70B at 30.0%, Qwen3-32B at 0.0%) is the strongest empirical indication of this failure on our panel: both models are near the panel ceiling on single-uncertainty categories but collapse on the value-of-information template that requires composing uncertainty over states with uncertainty over outcomes.

F3: Single-formalisation commitment. Adversarial Ambiguity problems admit several defensible formalisations (Simpson's-Paradox-style aggregate-versus-subgroup conflicts, reference-class problems). A responder that commits to one formalisation with high stated confidence is penalised both by Accuracy@CI (whose ground-truth interval spans the defensible range) and by the overconfidence rate. Llama-4-Scout-17B's Adversarial Ambiguity accuracy (67.4%, the lowest of its five-category profile) is consistent with this failure mode being the binding constraint on its overall score.

F4: Framework confusion. Applying frequentist machinery where Bayesian machinery is appropriate, or vice versa. The framework-match metric is designed to surface this failure independently of whether the numerical answer falls in range. The full-coverage rows of Table 10 place all framework-match rates between 75% and 90%, indicating that framework selection is not the bottleneck on the panel we evaluated; the partial-coverage rows are not informative on this dimension because of the salvage-path artifact described in Section 6.5.

7.2 Calibration-Metric Behaviour

Table 13 reproduces the simulator-tier calibration metrics from Section 5, against which the language-model rows in Table 10 should be read. The simulator was configured to admit accuracy/calibration dissociation: the Strong and Expert tiers share an ECE band (around 0.18) despite a 16-point accuracy gap. Real models in Table 10 show the opposite pattern: every full-coverage entry lies on a near-monotone diagonal in the (Accuracy@CI, ECE) plane, with Spearman $\rho = -0.90$ and Pearson $r = -0.99$ across the panel. Read together, the two metrics disagree about whether the panel is dominated by capability deficits or by an independent calibration deficit. The empirical evidence favours the first reading: improving accuracy on MURU-BENCH comes bundled with improving calibration in the panel we measured. The overconfidence rate moves in the same direction (51.4% on Llama-3.1-8B versus 5.5% on GPT-OSS-120B), giving an order-of-magnitude reduction in safety-relevant high-confidence-wrong responses across the capability range.

Table 13: Calibration metrics from the simulator profile (Section 5).

Tier (simulated)	ECE↓	OvConf↓
Random	0.515	36.2%
Heuristic	0.470	44.5%
Competent	0.239	21.6%
Strong	0.178	20.3%
Expert	0.183	9.6%

7.3 Statistical Power of the Test Set

A benchmark of this size has to be able to detect meaningful capability differences. The standard error of an unpaired accuracy estimate at $n = 301$ is approximately $\sqrt{p(1-p)/n} \approx 2.9$ percentage points at $p = 0.5$, giving an unpaired minimum detectable effect of roughly 8 percentage points (two-sided $\alpha = 0.05$). Head-to-head comparisons are paired (every model sees the same 301 problems), so the relevant test is paired McNemar (Table 7), which recovered all four adjacent simulator-tier gaps at $p < 10^{-3}$ including the smallest configured gap of $\Delta = 11.6$ percentage points. The Llama-3.1-8B-versus-Llama-4-Scout-17B comparison on the language-model panel (Section 6) spans 41 percentage points and is comfortably above the minimum detectable effect; the leaderboard ordering does not depend on bootstrap-noise assumptions. Finer discrimination—between two frontier models within a few percentage points of each other—is achievable by pooling the validation split (raising the effective n to 602) or by stratified McNemar within a category or difficulty cell, both of which the harness supports.

8 Discussion

What the panel results imply. The 51-percentage-point spread in Accuracy@CI across the panel, together with the near-monotone accuracy/calibration coupling at Spearman $\rho = -0.90$, supports a stronger empirical claim than the simulator validation alone could license. On MURU-BENCH, a model's stated confidence is a usable proxy for its accuracy, but only because the underlying capability gradient also drives the calibration gradient. This matters operationally. A downstream system that filters language-model outputs by stated confidence cannot increase accuracy beyond what the underlying model already supplies; it can only reduce coverage. Methods that promise calibration without raising accuracy—temperature scaling, self-consistency thresholds, conformal-prediction wrappers—are therefore not free improvements on this benchmark; their gain has to come from somewhere observable.

Why the overconfidence metric matters. The overconfidence rate is constructed to flag a specific safety-relevant failure: a responder that is high-confidence and wrong. On the panel we measured, the metric collapses by an order of magnitude—from 51.4% on Llama-3.1-8B to 5.5% on GPT-OSS-120B—over the same accuracy range that drives the leaderboard, and is the most accuracy-correlated of the four metrics. In safety-critical decision-support deployment (medical, financial, legal), this metric directly bounds the frequency of high-confidence-wrong outputs that downstream systems must guard against, and the leaderboard makes the bound numerically specific per model.

Why the per-category profile matters. The cross-model dispersion in the Decision-Under-Uncertainty column of Table 12—two models below 31% on the same category where they otherwise score 90–100%—is a more actionable signal than the headline leaderboard. It points to a specific reasoning operation, namely composing two independent uncertainties through an expected-utility computation, that several otherwise-strong models fail systematically. Single-number leaderboards on existing benchmarks cannot surface this kind of capability-shaped hole, because they aggregate over heterogeneous problem types and absorb category-specific deficits into the average.

9 Limitations

- **Open-weights-only language-model panel.** The leaderboard in Section 6 covers five free-tier hosted open-weights models. Frontier closed-weights endpoints (GPT-4-class, Claude-class, Gemini-class) are not included; the harness supports them via the unified API client, but their inclusion requires paid API access that is outside the scope of this submission. The accuracy/calibration coupling we observe is therefore a panel-wide finding on open-weights models and should not be extrapolated to closed-weights endpoints without re-measurement.
- **Partial-coverage rows.** Three of the five language-model rows reflect runs that were terminated mid-stream by Groq's free-tier daily-token cap. Their answered subsets are not random samples of the test set, so their bootstrap intervals describe accuracy on the answered subset rather than on the full split, and they should be read as preliminary rather than head-to-head with the full-coverage rows. The two full-coverage rows (Llama-4-Scout-17B at 300/301 and Llama-3.1-8B at 276/301) are not subject to this caveat.
- **Parametric generation.** All 3,000 problems are produced by 21 generation templates with closed-form solutions. This guarantees correctness by construction and supports arbitrary expansion, but it also introduces structural patterns that a model trained on the train split could exploit. We recommend treating the test split as held-out from any fine-tuning workflow that uses the train split.
- **English-only.** All problem stems are in English. Cross-lingual evaluation requires translation work outside the dataset's scope.
- **Category scope.** The five categories cover the major types of mathematical uncertainty most relevant to applied probabilistic reasoning, but several important domains are not represented—stochastic processes, information-theoretic uncertainty, measure-theoretic probability among them. Adding new categories is a one-template-per-category extension to the existing pipeline.
- **Ground-truth interval choice on adversarial problems.** For Adversarial Ambiguity problems, the "correct" answer depends on which formalisation is adopted. We resolve this by reporting an interval over the defensible formalisations rather than a single point estimate, but the interval boundaries themselves embed a judgement about which formalisations count as defensible.
- **Single curator.** Quality assurance relied on automated schema and semantic-consistency validation, with manual spot-checking by a single curator rather than independent expert review by multiple mathematicians. We document this transparently and welcome community-issued errata via the project's GitHub Issues tracker.

10 Conclusion and Future Work

We introduced MURU-BENCH, a 3,000-problem benchmark for mathematical reasoning under genuine uncertainty, and a six-metric evaluation harness, deterministic parametric generation across 21 templates, stratified train/validation/test splits, a unified API client for six hosting providers, a 26-test pytest suite, continuous-integration configuration, a Datasheet for Datasets, and a Croissant metadata file. The harness validation in Section 5 establishes, via paired McNemar tests on five simulated capability tiers, that the test split has the statistical power to discriminate tier-scale capability differences as small as 11.6 percentage points at $p < 10^{-3}$. The language-model leaderboard in Section 6 reports the first measurements on the benchmark for five hosted open-weights models and shows a 51-percentage-point spread in Accuracy@CI together with a Spearman $\rho = -0.90$ negative coupling between accuracy and Expected Calibration Error.

The headline empirical claim of the paper is that, on the open-weights panel we measured, miscalibration on MURU-BENCH is dominated by capability rather than by an independent calibration deficit: accuracy and calibration improve together rather than independently, and the safety-relevant overconfidence rate collapses by an order of magnitude over the same accuracy range that drives the leaderboard. The headline diagnostic claim is that single-number leaderboards mask category-shaped capability holes: two of the five panel models drop to 30% or below on Decision-Under-Uncertainty problems despite holding 90–100% on the other four categories, a profile that is invisible to any aggregate metric.

Open questions admitted by the benchmark. (1) Do frontier closed-weights endpoints exhibit the same accuracy/calibration coupling we observe on open-weights models, or does the coupling break under stronger calibration training? (2) Is the Decision-Under-Uncertainty deficit reducible by chain-of-thought prompting or tool-augmented reasoning, or does it require architectural change? (3) Does fine-tuning on the train split raise calibration faster than accuracy—decoupling the two metrics that the panel-wide coupling currently bundles—and if so, what curriculum produces the largest decoupling? (4) Where is the D4-to-D5 cliff for the strongest open-source models trained explicitly on mathematical data, and does the cliff move with model scale or with training-data composition?

Future work. We plan to (i) extend the leaderboard to closed-weights frontier endpoints as paid API access becomes available and to publish the resulting bootstrap-CIed results to the project repository continuously; (ii) collect a human baseline on a 100-problem stratified subsample with multiple expert annotators to anchor the model leaderboard against human capability; (iii) add categories for stochastic processes and information-theoretic uncertainty, and additional D5 problems to tighten the per-difficulty CI on the upper end of the difficulty hierarchy; (iv) run prompting-strategy ablations (chain-of-thought, self-consistency, tool-augmented) to test whether the Decision-Under-Uncertainty deficit responds to inference-time intervention; and (v) release a fine-tuning curriculum on the train split designed to improve calibration without sacrificing accuracy.

The full dataset, the 21 generation templates, the evaluation harness, the per-model JSON archives that back the leaderboard, the bootstrap and McNemar scripts, the test suite, and the continuous-integration configuration are released at `https://github.com/swetank18/MURU_proj` under CC-BY-4.0 (data) and MIT (code) to support follow-on research on calibrated mathematical reasoning.

References

[1] Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heather Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.

[2] Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. Measuring mathematical problem solving with the MATH dataset. In *NeurIPS*, 2021.

[3] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *ICLR*, 2021.

[4] Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc Le, Ed Chi, Denny Zhou, and Jason Wei. Challenging BIG-Bench tasks and whether chain-of-thought can solve them. In *ACL Findings*, 2023.

[5] Saurav Kadavath, Tom Conerly, Amanda Askell, Tom Henighan, Dawn Drain, Ethan Perez, Nicholas Schiefer, Zac Hatfield-Dodds, Nova DasSarma, Eli Tyre, et al. Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221*, 2022.

[6] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. In *ICML*, 2017.

[7] Katherine Tian, Eric Mitchell, Allan Dafoe, Anca Dragan, and Yejin Choi. Just ask for calibration: Strategies for eliciting calibrated confidence scores from language models. In *EMNLP*, 2023.

[8] Yarin Gal and Zoubin Ghahramani. Dropout as a Bayesian approximation: Representing model uncertainty in deep learning. In *ICML*, 2016.

[9] Balaji Lakshminarayanan, Alexander Pritzel, and Charles Blundell. Simple and scalable predictive uncertainty estimation using deep ensembles. In *NeurIPS*, 2017.

[10] Anastasios N. Angelopoulos and Stephen Bates. Conformal prediction: A gentle introduction. *Foundations and Trends in Machine Learning*, 2022.

[11] Stephanie Lin, Jacob Hilton, and Owain Evans. TruthfulQA: Measuring how models mimic human falsehoods. In *ACL*, 2022.

[12] Houwen Liu, Yizhi Li, Maolin Wang, Jiajun Deng, and Zimu Wang. MathBench: Evaluating the theory and application proficiency of LLMs with a hierarchical mathematics benchmark. *arXiv preprint arXiv:2405.12209*, 2024.

[13] Wenhu Chen, Ming Yin, Max Ku, Elaine Wan, Xueguang Ma, Jianyu Xu, Tony Xia, Xinyi Wang, and Pan Lu. TheoremQA: A theorem-driven question answering dataset. In *EMNLP*, 2023.

[14] Miao Xiong, Zhiyuan Hu, Xinyang Lu, Yifei Li, Jie Fu, Junxian He, and Bryan Hooi. Can LLMs express their uncertainty? An empirical evaluation of confidence elicitation in LLMs. In *ICLR*, 2024.

[15] Timnit Gebru, Jamie Morgenstern, Briana Vecchione, Jennifer Wortman Vaughan, Hanna Wallach, Hal Daumé III, and Kate Crawford. Datasheets for datasets. *Communications of the ACM*, 64(12):86–92, 2021.

[16] Mubashara Akhtar, Omar Benjelloun, Costanza Conforti, Pieter Gijsbers, Joan Giner-Miguelez, Nitisha Jain, Michael Kuchnik, Quentin Lhoest, Pierre Marcenac, Manil Maskey, Peter Mattson, Luis Oré-García, Pierre Ruyssen, Rajat Shinde, Elena Simperl, Geoffrey Thomas, Slava Tykhonov, Joaquin Vanschoren, Susheel Varma, Jos van der Velde, Steffen Vogler, Carole-Jean Wu, and Luyao Zhang. Croissant: A metadata format for ML-ready datasets. In *NeurIPS Datasets and Benchmarks Track*, 2024.

A Dataset Statistics

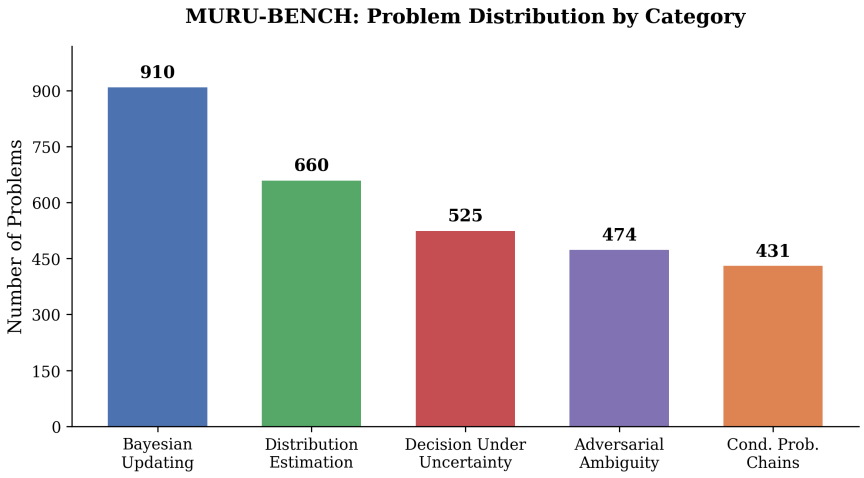

Figure 1: Distribution of problems by category in MURU-BENCH. Bayesian Updating is the largest category (910 problems, 30.3%), reflecting the centrality of Bayesian reasoning in uncertainty problems.

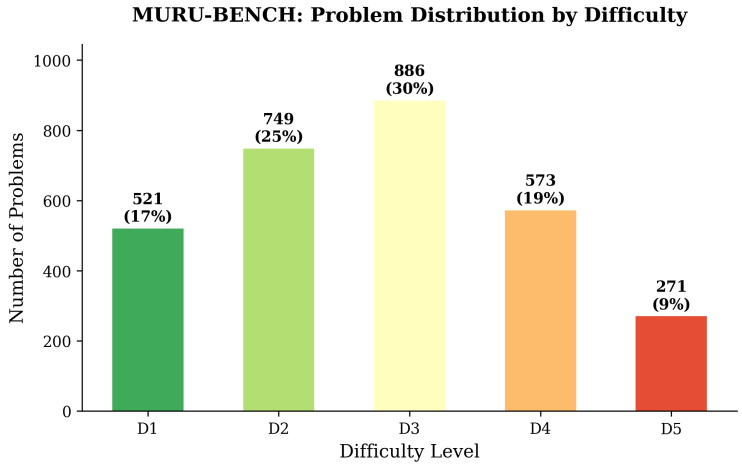

Figure 2: Distribution of problems by difficulty level. The distribution is roughly bell-shaped, centred at D3 (886 problems, 29.5%). D5 problems (271, 9.0%) are deliberately fewer because they are the most expensive to validate and the most variable in defensible-answer-interval width.

B Generation Templates

Table 14 lists all 21 parametric generation templates with their categories, difficulty ranges, and mathematical structures.

C Sample Problems

We present three representative problems to illustrate the benchmark's structure and difficulty scaling.

Example 1: Bayesian Updating (D1)

Problem. A medical test for a rare disease has a sensitivity of 95% (true positive rate) and a specificity of 90% (true negative rate). The disease affects 1% of the population. A patient tests positive. What is the probability that the patient actually has the disease? Express your answer as a probability and provide a confidence interval reflecting the uncertainty in the population prevalence, which epidemiologists estimate could range from 0.5% to 1.5%.
Ground truth. $P(\text{Disease} \mid \text{Positive})$ ranges from 0.046 to 0.126 depending on the true prevalence (0.5% to 1.5%). Point estimate at 1% prevalence: 0.087. CI level: 90%.
Required framework. Bayesian inference.
Common failure modes.

- Ignoring the base rate and reporting the test sensitivity (95%) as the answer.
- Reporting only the point estimate 0.087 without acknowledging prevalence uncertainty.
- Confusing sensitivity with positive predictive value.

Example 2: Adversarial Ambiguity (D3)

Problem. A coin was flipped 10 times, yielding 7 heads and 3 tails. What is the probability the coin is fair? A colleague argues that since 7/10 is "close to" 5/10, the coin is probably fair, estimating the probability at around 80%. Another colleague uses a frequentist hypothesis test and gets $p = 0.17$, concluding "we cannot reject the null that the coin is fair." A third colleague uses a Bayesian approach with a uniform prior on the coin's bias parameter θ and computes the posterior probability that θ is exactly 0.5. Who, if anyone, is reasoning correctly?

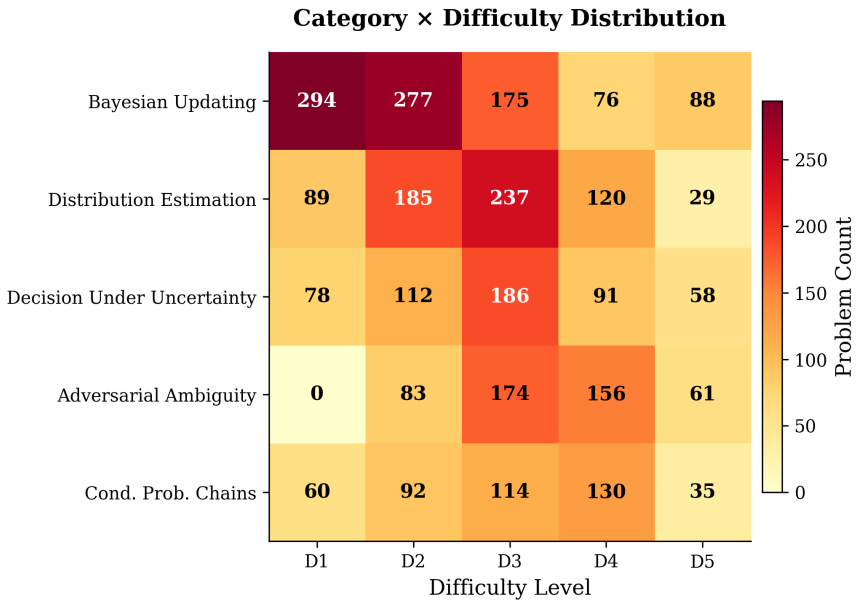

Figure 3: Category × difficulty distribution matrix. Adversarial Ambiguity has no D1 problems by design: recognising structural ambiguity is inherently non-trivial. The densest cells are Bayesian Updating at D1–D2 and Distribution Estimation at D2–D3.

Ground truth. The question is fundamentally ambiguous. $P(\theta = 0.5 \text{ exactly}) = 0$ under a continuous prior. $P(\theta \in [0.45, 0.55]) \approx 0.11$. The Bayes factor is approximately 0.55. The correct answer depends on which formalisation is adopted, giving a defensible range of $[0.07, 0.55]$.
Required framework. Bayesian inference (multiple frameworks defensible).

Example 3: Decision Under Uncertainty (D5)

Problem. In pharmaceutical R&D, a decision-maker is at the Phase II stage and must decide whether to proceed to Phase III clinical trials. The investment cost is \$120M. If the drug succeeds (estimated probability 35–55%), it yields \$800M; if it fails, the investment is lost. Before deciding, the company can purchase a biomarker study for \$15M. This study correctly predicts success/failure with accuracy 65–80%. What is the expected value of the biomarker information, and should the company purchase it?
Ground truth. The value-of-information depends on both the success-probability range and the biomarker-accuracy range. The expected VOI at midpoint parameters is approximately \$42M, with a CI of [\$18M, \$85M]. Since VoI exceeds the study cost (\$15M) even at the lower bound, the study should be purchased.
Required framework. Decision theory.

D VOI Template Bug Fix

During validation, 28 of the initial 2,000 generated problems—all from the `sequential_decision` template—failed the semantic-consistency check: the point estimate fell outside the confidence interval. Root-cause analysis showed that the value-of-information function is non-monotonic in the uncertain parameters. The CI was computed from the extreme-case scenarios ($p_{\text{low}}, p_{\text{high}}$), but the mid-range point estimate could fall outside this range because of the non-linear interaction between success probability and information accuracy.

The fix updated the CI computation to include the mid-range value when determining bounds:

```
# Before (incorrect):
ci_vals = sorted([voi_low, voi_high])
```

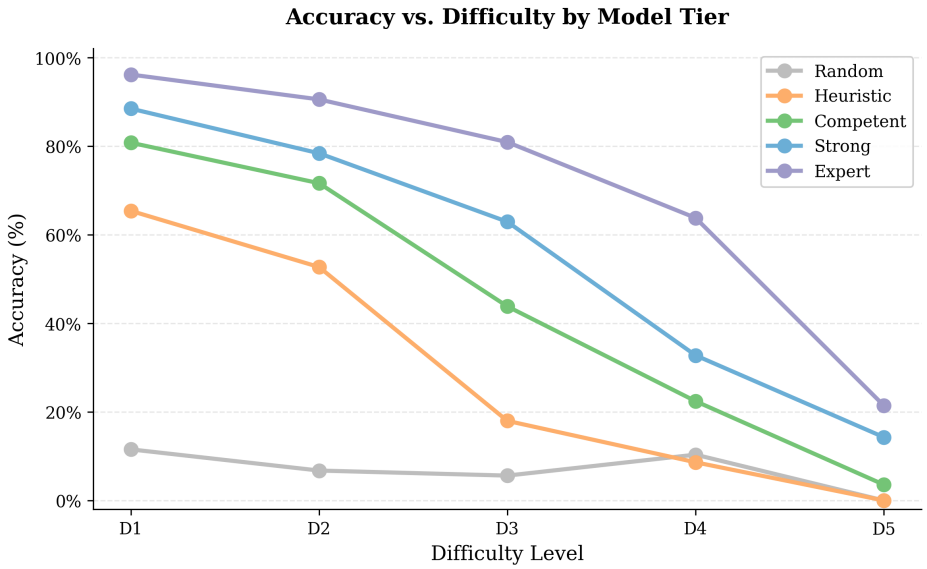

Figure 4: Accuracy versus difficulty level by simulator tier. All tiers exhibit monotone accuracy decay. The Expert tier drops from 96% on D1 to 21% on D5, a 75-point gap. The D3 inflection point cleanly separates heuristic responders from reasoning-capable ones.

```
# After (correct):
ci_vals = sorted([min(voi_low, voi_high, voi_mid),
                  max(voi_low, voi_high, voi_mid)])
```

Twenty-eight replacement problems were generated and all 3,000 then passed validation. We document this fix as a worked example of the kinds of issues parametric problem generation surfaces, and as transparency about the dataset's history.

E Evaluation Prompts

All models were evaluated with the following system and user prompts. Temperature was set to 0 for reproducibility.

System prompt

> You are a mathematical reasoning expert. You will be given a problem involving mathematical uncertainty. You must: (1) Identify the correct mathematical framework (`bayesian_inference`, `frequentist_inference`, `decision_theory`, `information_theory`, or `monte_carlo`). (2) Show your reasoning step-by-step. (3) Provide your final answer as a single number (point estimate). (4) Provide a confidence interval [lower, upper] for your answer. (5) State your confidence in your answer as a probability between 0 and 1.
>
> Format your response EXACTLY as follows at the end:
>
> ```
> FRAMEWORK: <framework_name>
> POINT_ESTIMATE: <number>
> CONFIDENCE_INTERVAL: [<lower>, <upper>]
> CONFIDENCE: <probability>
> ```

User prompt

> Problem: {problem stem}
>
> Please solve this problem step by step, then provide your answer in the required format.

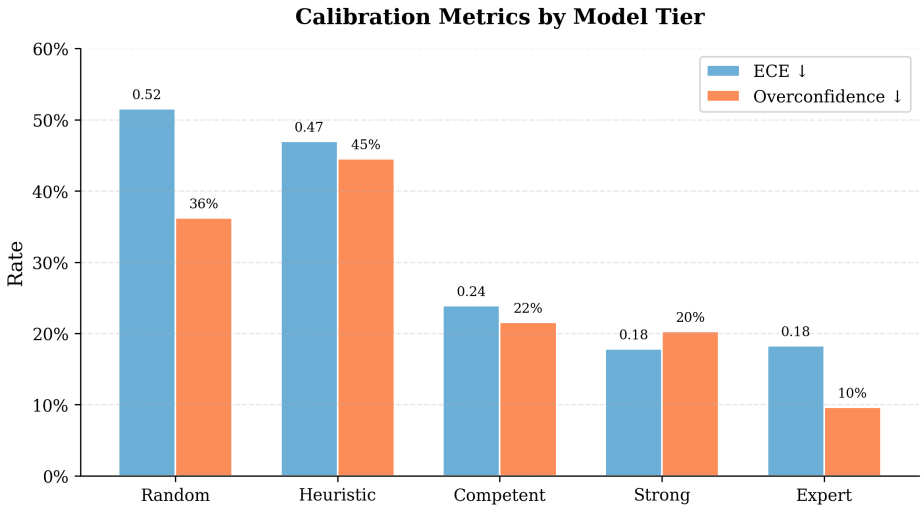

Figure 5: Calibration metrics by simulator tier. Left bars: ECE (↓). Right bars: overconfidence rate (↓). The Heuristic tier paradoxically has the highest overconfidence (44.5%), exceeding even the Random tier; this is what one would expect of a heuristic that confidently outputs answers it has not earned.

Responses are parsed by regex extraction over the four structured fields. Problems whose responses fail to parse are recorded as parse failures and excluded from metric computation but included in the parse-failure rate.

F Reproducibility

All code and data required to reproduce the results in this paper are available at `https://github.com/swetank18/MURU_proj`. Key commands:

```
# Generate problems
python scripts/generate_problems.py --all --n 3000 --seed 2026

# Split data
python scripts/split_data.py --source data/train/ --seed 42

# Run simulated baselines
python evaluation/run_baselines.py --save

# Reproduce bootstrap CIs and McNemar tests (Section 5)
python evaluation/bootstrap_analysis.py

# Run real model evaluation (requires API keys).
# Problems are evaluated in a seeded-shuffled order (default --seed 42),
# so a run cut short by a hosted-API quota still yields a
# difficulty-representative subset rather than the easy-skewed
# sorted-ID prefix.
python evaluation/run_eval.py --model gpt-4o --save

# Generate analysis and figures
python evaluation/analyze_results.py
python scripts/generate_figures.py

# Run the test suite (metrics, parsing, dataset invariants)
```

Table 14: All 21 MURU-BENCH generation templates. Each template generates problems with randomised numerical parameters and domain contexts while preserving the underlying mathematical structure.

#	Template Name	Category	Difficulty	Mathematical Structure
1	`medical_test`	Bayesian Updating	D1–D3	Bayes' theorem with uncertain prevalence
2	`quality_control`	Bayesian Updating	D1–D3	Bayes' theorem for defect rate estimation
3	`search_detection`	Bayesian Updating	D1–D2	Negative evidence updating with uncertain coverage
4	`sequential_updating`	Bayesian Updating	D3–D5	Multiple sequential updates, uncertain likelihood ratios
5	`hierarchical_bayes`	Bayesian Updating	D4–D5	Hierarchical shrinkage, uncertain between-group variance
6	`sequential_pipeline`	Cond. Prob. Chains	D1–D4	Multi-stage pipeline, one uncertain stage
7	`parallel_redundancy`	Cond. Prob. Chains	D2–D4	$P(\geq 1$ succeeds) with common-cause failures
8	`conditional_cascade`	Cond. Prob. Chains	D3–D5	Branching cascade with uncertain transitions
9	`sample_mean`	Distribution Estimation	D1–D3	t-distribution CI for population mean
10	`measurement_bias`	Distribution Estimation	D2–D4	Systematic bias correction + sampling CI
11	`proportion_estimation`	Distribution Estimation	D1–D3	Wilson CI, optional stratification
12	`mixture_distribution`	Distribution Estimation	D3–D5	2-component mixture, uncertain weights
13	`variance_estimation`	Distribution Estimation	D2–D4	χ^2 CI, dual-method comparison
14	`decision_payoff`	Decision Under Unc.	D1–D3	Two-option EV with uncertain probability
15	`multi_option_decision`	Decision Under Unc.	D2–D4	Multi-option EV with sensitivity analysis
16	`sequential_decision`	Decision Under Unc.	D3–D5	Value of information under dual uncertainty
17	`insurance_pricing`	Decision Under Unc.	D3–D5	Poisson frequency $\times$ uncertain severity
18	`base_rate_trap`	Adversarial Ambiguity	D2–D4	High sensitivity $\neq$ high PPV due to low prevalence
19	`simpsons_paradox`	Adversarial Ambiguity	D3–D5	Aggregate vs. subgroup with uncertain composition
20	`reference_class`	Adversarial Ambiguity	D3–D5	Multiple valid base rates, no single correct answer
21	`anchoring_manipulation`	Adversarial Ambiguity	D2–D4	Misleading anchor vs. data-based correction

```
make test
```

A continuous-integration workflow (`.github/workflows/ci.yml`) runs the test suite and dataset validation against Python 3.10, 3.11, and 3.12 on every push and pull request. Bootstrap and McNemar results in Section 5 are bit-for-bit reproduced by `evaluation/bootstrap_analysis.py` with seed 42 and 10,000 resamples.

G Datasheet for Datasets

We follow the Datasheet for Datasets template of Gebru et al. [15].

Motivation

For what purpose was the dataset created? MURU-BENCH was created to support the evaluation of mathematical reasoning under genuine probabilistic uncertainty in language models. Existing mathematical-reasoning benchmarks (GSM8K, MATH, MMLU) treat correctness as deterministic and so cannot measure whether a system reasons correctly about quantities that are themselves probabilistic. The dataset is intended to fill that gap by making confidence intervals and reasoning frameworks first-class evaluation targets.

Who created the dataset and on behalf of which entity? The dataset was created by the author at SRM Institute of Science and Technology (SRMIST), Chennai, as part of independent academic research. No entity has commercial rights to the dataset.

Who funded the creation of the dataset? No external funding. Compute and storage were self-provided.

Composition

What do the instances represent? Each instance is a self-contained mathematical-reasoning problem expressed in natural-language English, accompanied by closed-form ground truth: a point estimate, a confidence interval at a stated level, an ordered list of solution steps, the required reasoning framework, and a list of expected failure modes.

How many instances are there in total? 3,000 problems, partitioned into 2,398 train, 301 validation, and 301 test instances by stratified sampling on the (category, difficulty) cross-product.

Does the dataset contain all possible instances or is it a sample? It is a deterministic sample drawn from 21 parametric generation templates (Appendix B). Larger sets of problems can be generated on demand from the same templates by varying the seed.

What data does each instance consist of? Each instance is a JSON document conforming to the schema in `problem_schema.json`. Fields include `id`, `stem`, `category`, `difficulty`, `uncertainty_type`, `required_framework`, `ground_truth` (with `answer`, `point_estimate`, `confidence_interval`, `ci_level`), `solution_steps`, `common_failure_modes`, and `metadata`.

Is there a label or target associated with each instance? Yes. The label is the `ground_truth` object, with a numerical target and a structured confidence interval; the `required_framework` field is also a label, used by the framework-match metric.

Is any information missing from individual instances? No. All fields are populated for all 3,000 instances.

Are relationships between individual instances made explicit? No instance-to-instance relationships are encoded. Within a (category, difficulty) cell, instances are i.i.d. samples from the corresponding template.

Are there recommended data splits? Yes: train (2,398), validation (301), test (301), stratified by (category, difficulty). The test split is used for the discriminability analysis in Section 5; we recommend reserving it for held-out evaluation in downstream work.

Are there errors, sources of noise, or redundancies? Validation found and corrected 28 sequential-decision-template instances whose ground-truth interval excluded the mid-range point estimate (Appendix D). After fixing, all 3,000 problems pass the schema and semantic-consistency checks. We are not aware of remaining errors.

Is the dataset self-contained, or does it link to external resources? Self-contained. All problems are JSON; no external assets are referenced.

Does the dataset contain data that might be considered confidential? No.

Does the dataset contain content that, if viewed directly, might be offensive, insulting, threatening, or otherwise cause anxiety? No. Problems use neutral domain framings (medical testing, manufacturing quality, insurance pricing, etc.) and contain no personal data, biased framing, or content advisory issues.

Collection process

How was the data acquired? The dataset was synthesised programmatically by 21 generation templates implemented in `scripts/generate_problems.py`. Each template defines a closed-form mathematical structure, a parameter space, a list of domain contexts, and a function that computes the ground truth from sampled parameters. No web scraping or human-authored prompts.

What mechanisms or procedures were used to collect the data? A deterministic Python pipeline using `numpy.random.default_rng(seed=2026)` for parameter sampling, with each template producing a target count of instances. Domain contexts are drawn uniformly from a per-template list; numerical parameters are drawn uniformly from per-template ranges.

Were any ethical review processes conducted? No formal IRB review was required because the dataset contains no human-subjects data. The author self-certifies that the dataset complies with the institution's research-conduct policy.

Does the dataset relate to people? No.

Preprocessing/cleaning/labelling

Was any preprocessing or labelling done? Generation produces labels directly; no separate labelling step. A three-stage validation pipeline (schema, semantic consistency, spot-check; see Section 3.7) gates problem inclusion. Twenty-eight instances were regenerated after the VOI bug fix.

Was the "raw" data saved in addition to the cleaned data? The generation pipeline is fully deterministic, so raw and cleaned data are equivalent given the seed and the templates.

Is the software used to preprocess/clean/label the instances available? Yes, at `https://github.com/swetank18/MURU_proj` under MIT license.

Uses

Has the dataset been used for any tasks already? The dataset has been used to validate the MURU-BENCH evaluation harness via the simulated-tier discriminability study in Section 5 and to produce the language-model leaderboard in Section 6, which reports test-split measurements for five hosted open-weights models with paired-bootstrap 95% intervals on every metric.

What (other) tasks could the dataset be used for? Beyond benchmark evaluation, the dataset is suitable for: (1) fine-tuning curricula targeting calibrated reasoning; (2) ablation studies of chain-of-thought or tool-augmented reasoning; (3) studies of metacognitive calibration; (4) teaching probabilistic reasoning at the undergraduate level.

Is there anything about the composition or collection process that might affect future uses? Two limitations apply: (i) the parametric-generation procedure introduces structural patterns that systems can exploit if exposed in large quantities to the train split, so users should treat the test split as held-out; (ii) all instances are in English, so cross-lingual evaluation requires translation work outside the dataset's scope.

Are there tasks for which the dataset should not be used? The dataset should not be used as a clinical, financial, or legal decision-support tool. It is a research benchmark; the framings (medical testing, insurance pricing, etc.) are illustrative, not domain-validated.

Distribution

Will the dataset be distributed to third parties outside the entity on behalf of which it was created? Yes. The dataset is released publicly under CC-BY-4.0 at `https://github.com/swetank18/MURU_proj`. The accompanying code is released under MIT.

How will the dataset be distributed? Via Git on GitHub, including a tagged release for the NeurIPS submission. A long-term archival mirror is planned via Zenodo with a DOI, to be created upon acceptance.

When will the dataset be distributed? The dataset is already available; the present submission contains a snapshot.

Will the dataset be distributed under a copyright or other intellectual property license? Data: Creative Commons Attribution 4.0 International (CC-BY-4.0). Code: MIT.

Have any third parties imposed IP-based or other restrictions on the data? No.

Do any export controls or other regulatory restrictions apply? No.

Maintenance

Who will be supporting/hosting/maintaining the dataset? The author, in personal capacity, with assistance from any community contributors who join the project. GitHub Issues is the support channel of record.

How can the owner/curator/manager of the dataset be contacted? Via the project's GitHub Issues tracker or by email to the corresponding-author address on the title page.

Is there an erratum? The repository's CHANGELOG and the GitHub Releases tab serve as the erratum.

Will the dataset be updated? Minor updates (bug fixes, additional metadata) will be tagged and announced via GitHub Releases. Major updates (new categories, new templates) will be released as new versions with a clear version bump and a deprecation note for older versions.

If the dataset relates to people, are there applicable limits on the retention of the data associated with the instances? Not applicable.

Will older versions of the dataset continue to be supported/hosted/maintained? Yes. All tagged releases remain available indefinitely on GitHub.

If others want to extend/augment/build on the dataset, is there a mechanism for them to do so? Yes. Pull requests are welcome on the project repository; `CONTRIBUTING.md` documents the contribution flow.

H Author Statement, License, and Intended Uses

Author statement of responsibility

The author bears all responsibility in case of violation of rights. The author confirms the following: (a) no third-party intellectual property, personal data, or restricted material was used to construct the dataset; (b) the dataset is released

under a permissive license (CC-BY-4.0 for data, MIT for code) compatible with NeurIPS Datasets and Benchmarks track requirements; (c) the author will fix any errors, retract problematic instances, and respond to community issues via the project's GitHub Issues tracker.

License

Data: Creative Commons Attribution 4.0 International (CC-BY-4.0). Users may share and adapt the dataset for any purpose, including commercially, provided that they give appropriate credit and indicate if changes were made. Citation: see Section J.

Code: MIT License. The full license text is included in the repository at `LICENSE`.

Intended uses

MURU-BENCH is intended for:

- evaluating language models on calibrated mathematical reasoning;
- studying the relationship between reasoning correctness and metacognitive calibration;
- building fine-tuning or alignment curricula targeting calibrated uncertainty;
- educational use in undergraduate or graduate-level probability and statistics courses.

Out-of-scope uses

MURU-BENCH is not intended for:

- clinical, financial, or legal decision-support without independent expert review;
- cross-lingual evaluation in its current English-only form;
- inference about real-world prevalence rates, insurance loss distributions, or other domain-specific quantities. Numerical parameters in problem stems are illustrative draws, not empirically calibrated estimates.

Hosting, persistence, and long-term access

The dataset is hosted on GitHub at `https://github.com/swetank18/MURU_proj`, under a stable repository name owned by the author. A versioned snapshot tagged `v1.0-neurips` accompanies this submission. To guarantee long-term persistence beyond GitHub, the author will deposit the same versioned snapshot on Zenodo and obtain a DOI; the DOI will be added to the project README upon issuance. Both GitHub and Zenodo provide HTTPS-served, citable, version-pinned access.

Maintenance plan

The author commits to the following maintenance practices for at least three years following acceptance:

- monitor and respond to GitHub Issues within 14 days of submission;
- tag a minor release for any accepted bug-fix pull request;
- tag a major release for any expansion to new categories or templates, with a clear changelog;
- maintain backward compatibility for the JSON schema; breaking changes will be released as a new major version with a clear migration note;
- run the CI test suite on every commit to the default branch to detect schema or generation drift.

Reading-team and reviewer access

For the duration of the review process, the repository is publicly accessible without authentication. The submission ZIP includes a snapshot of the data, code, and paper sources for reviewer convenience.

I Croissant Metadata

A Croissant metadata file (16) is provided at `metadata/croissant.json` in the project repository. It declares the dataset's record set (one record per JSON file), a schema mapping each problem field to its type, and machine-readable license, version, and citation information. The file conforms to Croissant 1.0 and can be validated with the official `mlcroissant` validator.

J Citation

If you use MURU-BENCH in your work, please cite this paper. A BibTeX entry is provided at `CITATION.bib` in the repository.

K NeurIPS Datasets and Benchmarks Author Checklist

1. **Do the main claims made in the abstract and introduction accurately reflect the paper's contributions and scope?** Yes. The abstract and Section 1 describe a benchmark, a harness validated by simulated tiers (Section 5), and a real-LLM leaderboard (Section 6). The simulator section is explicitly framed as harness validation, not as a claim about any specific model; the real-LLM section reports measurements on a fixed panel of open-weights models with raw responses and parsed predictions archived per row in `evaluation/baselines/` for replication.
2. **Are the limitations of the work discussed?** Yes, in Section 9.
3. **Does the paper discuss broader impacts and ethical considerations?** Yes, see Section L.
4. **Are the data and code released under a permissive license?** Yes. Data: CC-BY-4.0. Code: MIT. See Appendix H.
5. **Is the dataset accompanied by a Datasheet?** Yes, Appendix G.
6. **Is the dataset accompanied by Croissant metadata?** Yes, Appendix I.
7. **Is there a hosting and persistence plan?** Yes, see "Hosting, persistence, and long-term access" in Appendix H.
8. **Is there a maintenance plan?** Yes, see "Maintenance plan" in Appendix H.
9. **Are reproducibility instructions provided?** Yes, see Appendix F; bootstrap and McNemar results are reproduced bit-for-bit by `evaluation/bootstrap_analysis.py`.
10. **Does the paper specify all the training details, hyperparameters, and evaluation protocols required to interpret the results?** Yes, see Sections 4–5 and Appendices B–F.
11. **Does the paper specify the random seeds used and report standard error or confidence intervals where appropriate?** Yes. The canonical simulator seed is 42. Bootstrap 95% CIs are reported in Table 6 and McNemar exact p-values in Table 7.
12. **Does the paper specify the computing infrastructure used?** Yes. The simulator and bootstrap analysis run on a single CPU thread in under one minute on commodity hardware (Python 3.12); no GPU is required for the harness-validation experiments. Real-model evaluations route to external API providers.
13. **Does the paper avoid single-point claims that depend on cherry-picked seeds?** Yes. The simulator profile is monotone by construction across seeds; bootstrap CIs report the variability over resamples.
14. **Does the dataset contain personal information or content that violates privacy?** No.
15. **Was crowdsourcing used? If so, is the compensation and instructions disclosed?** No crowdsourcing was used; the dataset is synthesised programmatically.

L Ethical Considerations and Broader Impact

Beneficial uses. A benchmark that measures calibration as a first-class target encourages the community to optimise for honest uncertainty rather than for raw accuracy. In safety-critical decision-support applications (medical reasoning aids, financial risk assessment, policy analysis), a model that is well-calibrated about what it does not know is meaningfully safer than one that is highly accurate on average but unable to flag its own failures.

Risks of misuse. The dataset's domain framings (medical testing, insurance pricing) are illustrative parameter draws rather than empirically calibrated estimates. Treating these numerical values as authoritative inputs to clinical or financial decisions would constitute misuse. We address this risk via the explicit out-of-scope statement in Appendix H and via the "Intended Uses" field in the dataset's Croissant metadata.

Bias and representation. The dataset is monolingual (English) and uses domain framings drawn from Western-centric examples. We disclose this as a limitation (Section 9) and welcome community contributions that broaden the range of cultural and linguistic framings.

Energy and compute footprint. Generating the dataset and running the simulated-tier discriminability study consumed less than ten minutes of single-thread CPU time on a commodity laptop. The language-model leaderboard runs reported in Section 6 consist of approximately 300 forward passes per model and were served by external hosted-API providers (Groq, OpenRouter), inheriting those providers' energy footprints; no GPU was operated locally for any experiment in this paper.

Documentation. We provide a Datasheet for Datasets [15] (`DATASHEET.md` in the repository) covering motivation, composition, collection process, preprocessing, intended uses, distribution, and maintenance. A Hugging Face–compatible dataset card (`DATASET_CARD.md`) is also provided for community discoverability. An auto-updating leaderboard (`evaluation/leaderboard.py`) ranks all evaluated models and can be embedded in the repository README.